

DOKUMENTASI USER INTERFACE PUNCAK MAS APP

(Pemrograman Berorientasi Service IF 23 E)

Oleh:

Anisa Zeli Windiantika

NPM 2332106



PRODI INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS TEKNOKRAT INDONESIA

2026

DOKUMENTASI FRONTEND_USER

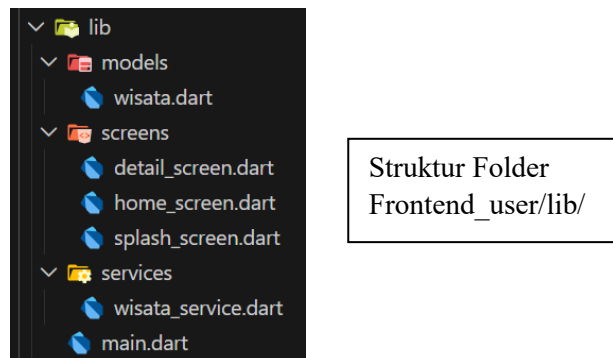
1. Teknologi yang Digunakan

Proyek ini dibangun menggunakan teknologi berikut:

- **Flutter**: Framework utama untuk pengembangan aplikasi mobile lintas platform (Android dan iOS).
- **Dart**: Bahasa pemrograman yang digunakan dalam Flutter untuk menulis kode aplikasi.
- **HTTP Package**: Library untuk melakukan permintaan HTTP ke API eksternal guna mengambil data wisata dan transaksi.
- **Material Design**: Sistem desain dari Google yang digunakan untuk menciptakan antarmuka pengguna yang konsisten dan modern.

2. Struktur Folder

Struktur folder proyek ini diorganisir secara sistematis untuk memudahkan pengembangan dan pemeliharaan kode. Berikut adalah penjelasan folder penting dalam direktori `lib/`:



- **screens/**: Berisi file-file yang mendefinisikan tampilan utama aplikasi, seperti halaman home, detail wisata, dan splash screen. Setiap screen merupakan komponen utama yang menampilkan konten kepada pengguna.
- **models/**: Menyimpan kelas-kelas data model, seperti `wisata.dart`, yang merepresentasikan struktur data wisata yang diambil dari API. Model ini membantu dalam pengelolaan data secara terstruktur.

```

1 // lib/models/wisata.dart
2 // Model data yang disesuaikan dengan response dari API NestJS (GET /wisata)
3
4 class Wisata {
5   final int id;
6   final String tanggal;
7   final String nama;
8   final int jumlah;
9   final int hargaTiket;
10  final int total;
11
12  Wisata({
13    required this.id,
14    required this.tanggal,
15    required this.nama,
16    required this.jumlah,
17    required this.hargaTiket,
18    required this.total,
19  });
20
21  /// Factory constructor: ubah JSON dari API menjadi objek Wisata
22  factory Wisata.fromJson(Map<String, dynamic> json) {
23    return Wisata(
24      id: json['id'] as int,
25      tanggal: json['tanggal'] as String,
26      nama: json['nama'] as String,
27      jumlah: (json['jumlah'] as num).toInt(),
28      hargaTiket: (json['hargaTiket'] as num).toInt(),
29      total: (json['total'] as num).toInt(),
30    );
31  }
32
33  /// Ubah objek Wisata kembali ke Map (jika diperlukan)
34  Map<String, dynamic> toJson() {
35    return {
36      'id': id,
37      'tanggal': tanggal,
38      'nama': nama,
39      'jumlah': jumlah,
40      'hargaTiket': hargaTiket,
41      'total': total,
42    };
43  }
44 }
45

```

Isi file wisata.dart yang mempresentasikan struktur data wisata dari API

- **services/**: Berisi layanan untuk interaksi dengan API, seperti `wisata\_service.dart`, yang menangani permintaan HTTP untuk mengambil atau mengirim data ke server.

```

1 // lib/services/wisata_service.dart
2
3 import 'dart:convert';
4 import 'package:flutter/foundation.dart';
5 import 'package:http/http.dart' as http;
6 import '../models/wisata.dart';
7
8 class WisataService {
9   /// Base URL dinamis sesuai platform
10  static String get _baseUrl {
11    if (kIsWeb) {
12      return 'http://localhost:3000';
13    } else {
14      return 'http://10.0.2.2:3000'; // Android Emulator
15    }
16  }
17
18  /// GET: Ambil semua data wisata
19  Future<List<Wisata>> fetchAllWisata() async {
20    final Uri url = Uri.parse('$_baseUrl/wisata');
21
22    try {
23      final response = await http.get(url).timeout(const Duration(seconds: 10));
24
25      if (response.statusCode == 200) {
26        final List<dynamic> jsonList = jsonDecode(response.body);
27        return jsonList.map((json) => Wisata.fromJson(json)).toList();
28      } else {
29        throw Exception(
30          'Gagal memuat data. Status: ${response.statusCode}, Body: ${response.body}',
31        );
32      }
33    } catch (e) {
34      throw Exception('Koneksi gagal (fetchAllWisata): $e');
35    }
36  }
37

```

Potongan code file wisata_service.dart untuk mengambil data

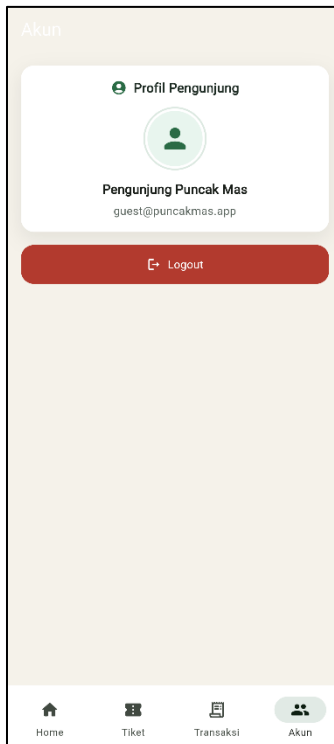
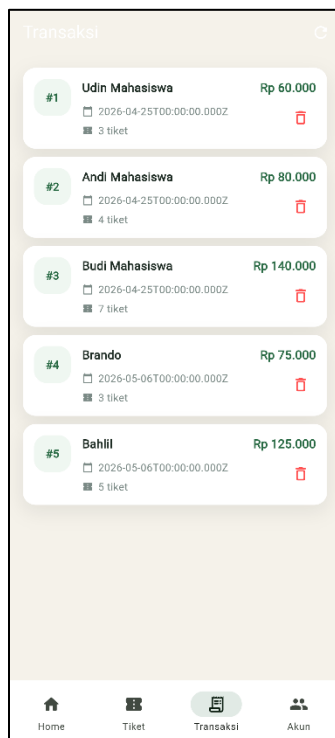
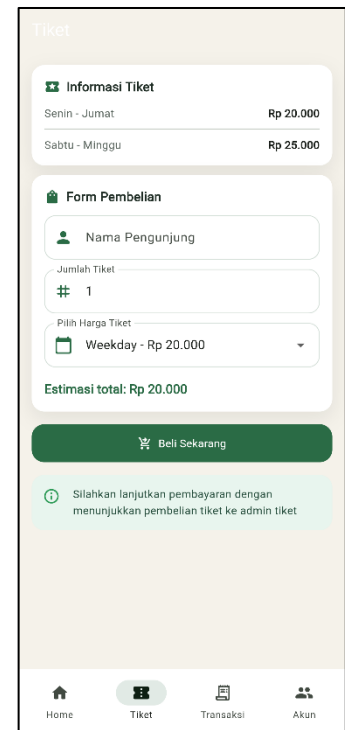
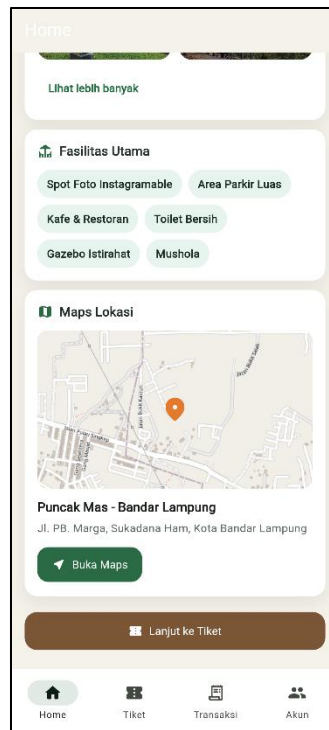
Struktur ini memastikan kode tetap terorganisir, mudah dibaca, dan dapat dikembangkan oleh tim pengembang.

3. Fitur Aplikasi

Aplikasi ini menyediakan berbagai fitur untuk mendukung pengalaman wisata pengguna:

- **Home Wisata:** Halaman utama yang menampilkan daftar destinasi wisata Puncak Mas dengan informasi singkat.
- **Informasi Wisata:** Detail lengkap tentang suatu destinasi, termasuk deskripsi, harga tiket, dan fasilitas.
- **Galeri Wisata:** Koleksi foto destinasi wisata yang ditampilkan dalam format grid, dengan opsi untuk melihat lebih banyak gambar.
- **Pembelian Tiket:** Fitur untuk memesan tiket wisata secara online melalui antarmuka sederhana.
- **Histori Transaksi:** Riwayat pembelian tiket pengguna, yang menampilkan data transaksi sebelumnya.
- **Maps Lokasi:** Integrasi peta untuk menunjukkan lokasi destinasi wisata (meskipun implementasi peta belum sepenuhnya selesai).
- **Bottom Navigation:** Navigasi bawah untuk berpindah antara halaman utama, transaksi, dan pengunjung dengan mudah.

Tampilan dari Fitur Aplikasi :



4. Penjelasan UI/UX

Desain UI/UX aplikasi ini mengutamakan kesederhanaan dan kemudahan penggunaan:

- **Clean UI:** Antarmuka yang bersih dan tidak berantakan, dengan fokus pada konten utama untuk menghindari kebingungan pengguna.
- **Responsive:** Desain yang menyesuaikan dengan berbagai ukuran layar perangkat mobile, memastikan pengalaman yang konsisten di Android dan iOS.
- **Modern Mobile Layout:** Menggunakan layout standar mobile dengan elemen seperti AppBar, BottomNavigationBar, dan Scaffold untuk navigasi yang intuitif.
- **Card-Based Design:** Informasi wisata ditampilkan dalam bentuk kartu (Card) yang mudah dibaca dan menarik secara visual.
- **Grid Galeri:** Galeri foto menggunakan GridView untuk menampilkan gambar dalam format grid yang rapi dan efisien.
- **Navigasi Sederhana:** Bottom navigation memungkinkan pengguna berpindah halaman dengan satu ketukan, tanpa perlu menu kompleks.

Konsep desain ini bertujuan untuk memberikan pengalaman pengguna yang nyaman dan efisien, sehingga pengguna dapat fokus pada eksplorasi wisata tanpa distraksi.

5. Penjelasan API Integration (singkat)

Aplikasi frontend ini terintegrasi dengan API backend untuk mengambil data wisata dan transaksi. Integrasi dilakukan menggunakan HTTP request melalui package 'http' di Dart. Data seperti daftar wisata, detail destinasi, dan histori transaksi diambil dari endpoint API dan ditampilkan secara real-time di aplikasi. Proses ini memastikan data selalu terkini tanpa perlu penyimpanan lokal yang kompleks.

6. Penjelasan State Management

State management dalam aplikasi ini menggunakan pendekatan sederhana dengan StatefulWidget dan metode 'setState'. StatefulWidget digunakan untuk komponen yang memerlukan perubahan state dinamis, seperti saat memuat data dari API atau memperbarui tampilan berdasarkan interaksi pengguna. Metode 'setState' dipanggil untuk memberitahu framework Flutter bahwa ada perubahan state, sehingga UI dapat diperbarui secara otomatis. Pendekatan ini cocok untuk aplikasi skala kecil seperti ini, meskipun untuk pengembangan lebih lanjut dapat dipertimbangkan penggunaan state management yang lebih advanced seperti Provider atau Riverpod.

7. Penjelasan Galeri Wisata

Fitur galeri wisata menampilkan koleksi foto destinasi menggunakan widget GridView, yang mengatur gambar dalam format grid responsif. Awalnya, hanya 4 gambar pertama yang ditampilkan untuk menghindari loading berat. Pengguna dapat melihat lebih banyak gambar dengan menekan tombol "lihat lebih banyak", yang akan memperluas grid secara dinamis. Implementasi ini mengoptimalkan performa aplikasi dengan lazy loading dan memastikan pengalaman visual yang menarik tanpa mengorbankan kecepatan.

8. Penjelasan Histori Transaksi

Fitur histori transaksi menampilkan daftar transaksi tiket yang telah dilakukan oleh pengguna. Data diambil langsung dari API backend dan ditampilkan dalam format list yang mudah dibaca. Fitur delete dihilangkan dari aplikasi user karena sesuai dengan role pengguna akhir, yang tidak diperbolehkan mengubah atau menghapus data transaksi. Hal ini memastikan integritas data dan keamanan aplikasi.

9. Cara Menjalankan Project

Untuk menjalankan proyek ini, ikuti langkah-langkah berikut:

1. Pastikan Flutter SDK telah terinstal di sistem Anda.
2. Buka terminal dan navigasi ke direktori proyek `Frontend\_user`.
3. Jalankan perintah `flutter pub get` untuk mengunduh semua dependensi yang diperlukan.
4. Jalankan perintah `flutter run` untuk menjalankan aplikasi di emulator atau perangkat fisik.
5. Pilih perangkat target (Android atau iOS) jika diminta.

Pastikan perangkat pengembangan terhubung dan API backend berjalan untuk pengalaman penuh.